

```
environment:
  - MYSQL_ROOT_PASSWORD=root
redis:
  image: redis
web:
  build: .
  command: bundle exec rails s -p 3000 -b '0.0.0.0'
volumes:
  - ./myapp
ports:
  - "3000:3000"
depends_on:
  - mysql
  - redis
```

In the web section, you can see ‘build’. which says, ‘build a container from the current working directory’; for MySQL and Redis, we need not build an image locally as we are using publicly available images. Build your environment with the ‘docker-compose build’ command:

```
<code:shell>
$ docker-compose build
docker-compose build
redis uses an image, skipping
mysql uses an image, skipping
Building web
Step 1: FROM ruby:2.3.1
2.3.1: Pulling from library/ruby
386a066cd84a: Already exists
75ea84187083: Pull complete
.
.
.
Step 8 : ADD . /app
----> 3a51a5637b94
Removing intermediate container 2f1b024e5daa
Successfully built 3a51a5637b94
```

The output makes it clear that it skips building an image for Redis and MySQL, and proceeds with web. As you can see, it starts with Step 1 and ends at Step 8. This is because we have specified eight commands in the Dockerfile. You will need to build whenever there is a change made to Gemfile or Dockerfile. Coming to the final step, let’s run our Dockerised Rails development environment:

```
<code:shell>
$ docker-compose up -d
Pulling redis (redis:latest)...
latest: Pulling from library/redis
386a066cd84a: Already exists
769149e3a45c: Pull complete
1f43b3c0854a: Pull complete
```

```
70e928127ad8: Pull complete
9ad9c0058d76: Pull complete
bc845722f255: Pull complete
105d1e8cd76a: Pull complete
Digest: sha256:c2ce5403bddabd407c0c63f67566fcab6facef90877de58
f05587cdf244a28a8
Status: Downloaded newer image for redis:latest
Creating railsredismysqldocker_redis_1
Creating railsredismysqldocker_mysql_1
Creating railsredismysqldocker_web_1
```

You have your application running and you can verify this by visiting <http://localhost:3000> in your browser. You might see a database error and that is because we need to set up a database for our application. Open a new terminal and run the following code:

```
<code:shell>
$ docker-compose run web rake db:create
Created database 'rails-redis-mysql-docker_development'
```

Here are a few commands that you might need while working:

```
<code:shell>
# Stop all running containers
$ docker-compose stop

# Destroy all containers, images, networks. You will have to
run rake db:create again if you do this.
$ docker-compose down

# Check logs
$ docker-compose logs web

# Validate docker-compose.yml. If there are no errors it will
display the contents of yaml.
$ docker-compose config
```

That’s it for this session. If by now you have doubts like ‘Why not add `rake db:create` to Dockerfile?’ or ‘Where is my data actually stored?’ then you are thinking along the right lines. 

References

- [1] <https://docs.docker.com/compose/overview/>
- [2] <https://docs.docker.com/engine/getstarted/>
- [3] <https://docs.docker.com/engine/tutorials/docker volumes/>
- [4] Rails demo project with Docker files: <https://gitlab.com/techmianack/rails-mrd-demo/tree/docker>

By: Abdul Karim Memon

The author has been a Linux user for the past seven years and has also helped Ola Cabs run DevOps. He is a FOSS evangelist and prefers Emacs as his IDE. He can be reached at abdulkarim@reversehack.in.